LAMBDA AND STREAM

CONTENTS OF THE CHAPTER

Lambda expressions and Streams, introduced in Java 8, are useful tools for reducing the number of lines of code needed, particularly when processing data. This chapter covers Functional Interfaces, Lambda expressions, Method References, Optionals, and especially how to use the Stream API.

9.1. FUNCTIONAL INTERFACE

A functional interface is a Java interface that contains exactly one abstract method and may also include multiple default or static methods.

```
@FunctionalInterface
public interface Flyable {
   void fly();

   default boolean alive() {
       return true;
   }
}
```

The **@FunctionalInterface** annotation is optional, but recommended to avoid errors (for example, when declaring more than one abstract method). Functional interfaces are also known as **SAM** (**Single Abstract Method**) interfaces.

In some programming languages, functions can be passed as input parameters to other functions, which provides flexibility. For example:

```
function printResult(calculate) {
   console.log('Result is', calculate())
}

printResult(function () {
   return 3.14
```

```
})
```

However, Java does not treat functions as first-class citizens. Parameters for a method in Java can only be basic data types or objects. In Java, methods cannot be passed directly as parameters like in some functional languages. Therefore, Java adopts an alternative approach to address this limitation. Instead of directly passing function B to function A, function B is "encapsulated" within an object (in the form of a functional interface). Then, that object is passed to function A. Function A simply retrieves and uses it.

```
// Define the template for the method passed in
@FunctionalInterface
interface Calculable {
    double calculate ();
}

public class Program {
    public static void printResult ( Calculable func) {
        System.out.println( "Result: " + func.calculate());
}
    public static void main ( String [] args) {
        printResult (new Calculable () {
            @Override
            public double calculate () {
return 3.14 ;
}
});
}
}
```

Output:

```
Result: 3.14
```

In a functional interface, you can declare default methods and static methods, which are methods that contain the keyword `default` or `static` and have a function body. For example:

```
@FunctionalInterface
interface Sayable {
   // default method
   default void say (){
       System.out.println( "Hello, this is default method" );
}
   // Abstract method
   void sayMore ( String msg);
   // static method
   static void sayLouder ( String msg){
System.out.println ( msg);
}
}
public class Program implements Sayable {
   public void sayMore ( String msg){ // implementing abstract
method
       System.out.println ( msg);
}
   public static void main ( String [] args) {
Program dm = new Program();
dm.say(); // calling default method
       dm .sayMore( "Work is worship" ); // calling abstract
method
Sayable .sayLouder( "Helloooo..." ); // calling static method
   }
}
```

9.2. LAMBDA EXPRESSIONS

First, we need to understand why we need to use Lambda expressions. The primary purpose of lambda expressions is to write more concise and expressive code. The syntax of a Lambda expression is as follows:

```
(argument-list) -> {body}
```

Consider the case where Lambda is not used:

```
interface Drawable {
```

```
void draw ();
}
```

There are two ways to create an object from the Drawable interface: the first is to create a Circle class that implements the Drawable interface;

```
class Circle implements Drawable {
   @Override
   public void draw () {
      System.out.println( "Circle" );
   }
}
public class Program {
   public static void main ( String [] args) {
      Circle c = new Circle();
      c.draw();
   }
}
```

The second method is to create the Circle object from the Drawable interface; in this case, you will need to implement the draw function when creating the object:

```
interface Drawable {
void draw ();
}

public class Program {
   public static void main ( String [] args) {
        Drawable circle = new Drawable () {
            @Override
            public void draw () {
            System.out.println( "Circle" );
         }
};
circle.draw();
}
}
```

Using Lambda expressions, the code can be shortened as follows:

```
@FunctionalInterface //It is optional
interface Drawable {
    void draw ();
}

public class Program {
   public static void main ( String [] args) {
Drawable circle = () -> System.out.println( "Circle" );
circle.draw();
}
}
```

If the draw function has one parameter, the syntax for passing the parameter
is as follows:

```
interface Drawable {
void draw (int r);
}

public class Program {
   public static void main ( String [] args) {
Drawable circle = r -> {
System.out.println( "Circle " + r);
};
circle.draw( 25 );
}
}
```

If the draw function has multiple parameters, the list of required parameters must be enclosed in parentheses:

```
interface Drawable {
   void draw ( int r, String color);
}

public class Program {
```

```
   public static void main ( String [] args) {
Drawable circle = (r, color) -> {
System.out.println( "Circle " + r + ", " + color);
};
circle.draw( 25 , "Blue" );
}
}
```

For functions with a return value, if the method body contains only one return statement, the keyword `return` can be omitted. For example:

```
interface Addable {
   int add ( int a, int b);
}

public class Program {
   public static void main ( String [] args) {
Addable addable = (a, b) -> a + b;
System.out.println(addable.add( 2 , 10 ));
}
}
```

In cases where the function body contains multiple statements, the body must be enclosed in curly braces, and the `return` keyword must be used explicitly.

Consider another example: in a foreach loop to iterate through a list, if not using Lambda, the forEach() method would need to create a Consumer object and write the following statement for the accept method:

```
public static void main ( String [] args) {
List<String> list = new ArrayList<>();
list.add( "Apple" );
list.add( "Banana" );
list.add( "Orange" );
list.add( "Grape" );
   list .forEach( new Consumer <String>() {
                    @Override
public void accept ( String s) {
```

```
System.out.println ( s);
}
}
);
}
```

However, when using Lambda, you only need to declare the Lambda expression as follows:

```
List<String> list = new ArrayList<>();
list.add( "Apple" );
list.add( "Banana" );
list.add( "Orange" );
list.add( "Grape" );
list.forEach(s -> System.out.println(s));
```

9.3. METHOD REFERENCES

Method references are used to refer to other methods when using Lambda expressions, which help make lambda expressions more concise.

9.3.1. References to static methods

To refer to a static method, use the following syntax:

```
ContainingClass::staticMethodName
```

For example, in the following case, the Lambda expression refers to the static method saySomething() of the class Program:

```
@FunctionalInterface //It is optional
interface Sayable {
    void say ();
}

public class Program {
public static void saySomething () {
        System.out.println("Hello, this is static method.");
}
```

```
public static void main ( String [] args) {
        Sayable sayable = () -> Program.saySomething ();
        sayable.say ();
}
}
```

Here, the Lambda expression can be shortened using static reference methods as follows:

```
@FunctionalInterface //It is optional
interface Sayable {
   void say ();
}

public class Program {
   public static void saySomething (){
       System.out.println( "Hello, this is static method." );
}
   public static void main ( String [] args) {
Sayable sayable = Program:: saySomething ;
sayable.say();
}
}
```

Consider another example: using method references to create two threads, thread1 and thread2, that perform the tasks written in the static methods runMethod1 and runMethod2:

```
public class Program {
   public static void runMethod1 () {
       System.out.println( "Thread is running... doing 123" );
}

   public static void runMethod2 () {
       System.out.println( "Thread is running... doing abc" );
}

   public static void main ( String [] args) {
```

```
Thread thread1 = new Thread(Program:: runMethod1 );
thread1.start();

Thread thread2 = new Thread(Program:: runMethod2 );
thread2.start();
}
}
```

9.3.2. Referencing a non-static method

Similar to static methods, instance methods can also be referenced using the following syntax:

```
containingObject::instanceMethodName
```

For example, after referencing the saySomething method of the Program class:

```
interface Sayable {
   void say ();
}

public class Program {
   public void saySomething () {
       System.out.println( "Hello, this is non-static method."
);
}

   public static void main ( String [] args) {
       // Referring non-static method using reference
       Sayable sayable = new Program()::saySomething;
       // Calling interface method
       sayable.say();
}
}
```

A similar example involves creating a thread to perform the printMsg() task declared in the Program class:

```
public class Program {
```

```
    public void printMsg () {
        System.out.println( "Hello, this is instance method" );
}

    public static void main ( String [] args) {
        Thread t = new Thread( new Program()::printMsg);
        t.start();
}
}
```

9.3.3. Referencing a constructor

To reference a constructor, use the following simple syntax, where only the keyword `new` is needed and the constructor name is omitted:

```
ClassName::new
```

For example, when creating a hello object, instead of writing the Lambda expression: msg -> new Message(msg), you can write it more concisely as follows:

```
interface Messageable {
    Message getMessage ( String msg);
}

class Message {
    Message (String msg) {
        System.out.print ( msg);
    }
}

public class Program {
    public static void main ( String [] args) {
        Messageable hello = Message:: new ;
        hello.getMessage( "Hello" );
}
}
```

9.4. PRE-DEFINED FUNCTIONAL INTERFACES

Java defines many functional interfaces that can be used in various situations; the following is a list of functional interfaces. All functional interfaces are presented in the java.util package. Functional interfaces can be divided into five groups:

- **Functions** accept arguments and returns a result.
- **Operators** return results of the same type as their operands (a special case of functions);
- **Predicates** accept arguments and return a boolean value (functions with boolean values);
- **Suppliers** do not accept arguments and returns a value;
- **Consumers** accept arguments but do not return a value.

Thanks to the naming conventions for functional interfaces in the java.util.function package, you can easily understand an interface characteristic simply by looking at its name prefix, specifically as follows:

- An operation accepts several parameters. The prefix **Bi** indicates that a function, a predicate, or a consumer accepts two parameters. Similarly to the prefix Bi, the prefixes Unary and Binary indicate that an operator accepts one and two parameters, respectively.
- An input parameter type. The prefixes **Double**, **Long**, **Int**, and **Obj** indicate the type of input value. For example, the IntPredicate interface represents a Predicate that accepts a value of type Integer.
- The prefixes **ToDouble**, **ToLong**, and **ToInt** indicate the type of output value. For example, the interface ToIntFunction<T> represents a function that returns a value of type Integer.

These interfaces are all defined in java.util.function:

Functional interface	Description
BiConsumer<T,U>	This represents an operation that accepts two input

Functional interface	Description
	arguments and returns no result.
Consumer<T>	It represents an operation that accepts a single argument and does not return a result.
Function<T,R>	It represents a function that accepts one argument and returns a result.
Predicate<T>	It represents a function that accepts one argument and returns a result that is true or false.
BiFunction<T,U,R>	This represents a function that accepts two arguments and returns a result.
BinaryOperator<T>	This represents an operation based on two operands of the same data type. It returns a result of the same type as the operands.
BiPredicate<T,U>	This represents a function that accepts two arguments and returns either true or false.
BooleanSupplier	Represents a function that provides results with boolean values.
DoubleBinaryOperator	It represents an operation based on two operands of type Double and returns a value of type Double.
DoubleConsumer	This represents an operation that accepts a single Double argument and returns no result.
DoubleFunction<R>	This represents a function that accepts a Double argument and produces a result.
DoublePredicate	Represents a predicate (a boolean function) of an argument of type Double.
DoubleSupplier	Representing a provider of Double results.
DoubleToIntFunction	This represents a function that accepts a Double argument and produces an Int result.
DoubleToLongFunction	This represents a function that accepts a Double argument and produces a Long result.
DoubleUnaryOperator	This represents an operation on an operand of type Double that returns a result of type Double.
IntBinaryOperator	It represents an operation based on two operands of

Functional interface	Description
	type Int and returns a result of type Int.
IntConsumer	It represents an operation that accepts a single integer argument and returns no result.
IntFunction<R>	This represents a function that accepts an integer argument and returns a result.
IntPredicate	It represents a predicate (a boolean function) of an integer argument.
IntSupplier	Represents an integer type provider.
IntToDoubleFunction	This represents a function that accepts an integer argument and returns a Double value.
IntToLongFunction	This represents a function that accepts an integer argument and returns a long value.
IntUnaryOperator	Represents an operation on a single integer operand that produces an integer result.
LongBinaryOperator	It represents an operation based on two operands of type Long and returns a result of type Long.
LongConsumer	This represents an operation that accepts a single Long argument and returns no result.
LongFunction<R>	This represents a function that accepts a Long argument and returns a result.
LongPredicate	Represents a predicate (a boolean function) of an argument of type Long.
LongSupplier	Represents a provider of Long-type results.
LongToDoubleFunction	This represents a function that accepts a Long argument and returns a Double result.
LongToIntFunction	This represents a function that accepts a Long argument and returns an integer result.
LongUnaryOperator	It represents an operation on a single Long operand and returns a Long result.
ObjDoubleConsumer<T>	This represents an operation that accepts an object and a Double argument and returns no result.
ObjIntConsumer<T>	This represents an operation that accepts an object and

Functional interface	Description
	an integer argument. It does not return a result.
ObjLongConsumer<T>	Representing an operation that accepts an object and a Long argument, it returns no result.
Supplier<T>	Represents a supplier of results.
ToDoubleBiFunction<T,U>	This represents a function that accepts two arguments and produces a result of type Double.
ToDoubleFunction<T>	This represents a function that returns a result of type Double.
ToIntBiFunction<T,U>	This represents a function that accepts two arguments and returns an integer.
ToIntFunction<T>	Represents a function that returns an integer.
ToLongBiFunction<T,U>	This represents a function that accepts two arguments and returns a result of type Long.
ToLongFunction<T>	This represents a function that returns a result of type Long.
UnaryOperator<T>	It represents an operation on a single operand that returns a result of the same type as its operand.

The BiConsumer interface accepts two input arguments and does not return any result. Here, two representative objects provide a functional method `accept(Object, Object)` to perform custom operations. Example of BiConsumer:

```
import java.util.function.BiConsumer ;
public class Program {
   static void ShowDetails (String name, Integer age){
        System.out.println(name+ " " +age);
}
   public static void main ( String [] args) {
        // Referring method
        BiConsumer < String , Integer > biCon = Program::
ShowDetails ;
       biCon.accept( "Rama" , 20 );
       biCon.accept( "Shyam" , 25 );
```

```
        // even lambda expression
        BiConsumer<String, Integer> biCon2 = (name,
age)->System.out.println(name+ " " +age);
        biCon2.accept( "Peter" , 28 );
}
}
```

Consider another example:

```
import java.util.HashMap ;
import java.util.Map ;
import java.util.function.BiConsumer ;

public class Program {
   static void ShowDetails (Map<Integer, String> map, String
mapName) {
        System.out.println( "----------" + mapName + "
records----------" );
map.forEach((key, val) -> System.out.println(key + " " + val));
  }

   public static void main ( String [] args) {
        Map < Integer , String > map = new HashMap< Integer ,
String >();
        map.put( 100 , "Mohan" );
        map.put( 110 , "Sujeet" );
        map.put( 115 , "Tom" );
        map.put( 120 , "Danish" );
        // Referring method
        BiConsumer < Map < Integer , String >, String > biCon =
Program:: ShowDetails ;
        biCon.accept(map, "Student" );
}
}
```

Each **function** takes a value as a parameter and returns a unique value. For example, Function<T, R> is a generic interface representing a function that accepts a value of type T and produces a result of type R. For example:

```
Function<String, Integer> converter = Integer::parseInt;
converter.apply("1000"); // the result is 1000 (Integer)

ToIntFunction<String> anotherConverter = Integer::parseInt;
anotherConverter.applyAsInt("2000"); // the result is 2000
(int)

BiFunction<Integer, Integer, Integer> sumFunction = (a, b) -> a
+ b;
sumFunction.apply(2, 3); //5 (Integer)
```

Each **operator** receives and returns values of the same type. For example, UnaryOperator<T> represents an operator that accepts a value of type T and produces a result of the same type T. For example:

```
UnaryOperator<Long> longMultiplier = val -> 100_000 * val;
longMultiplier.apply(2L); // the result is 200_000L (Long)

IntUnaryOperator intMultiplier = val -> 100 * val;
intMultiplier.applyAsInt(10); // 1000 (int)

BinaryOperator<String> appender = (str1, str2) -> str1 + str2;
appender.apply("str1", "str2"); // "str1str2"
```

Each **Predicate** takes a value as a parameter and returns true or false. For example, Predicate<T> is a generic interface representing a Predicate that accepts a value of type T and produces a boolean result. Example:

```
Predicate<Character> isDigit = Character::isDigit;
isDigit.test('h'); // false (boolean)

IntPredicate isEven = val -> val % 2 == 0;
isEven.test(10); // true (boolean)
```

Each **supplier** does not accept parameters and returns a single value. For example, Supplier<T> represents a supplier that does not accept arguments and returns a value of type T. Example:

```
Supplier<String> stringSupplier = () -> "Hello";
stringSupplier.get(); // the result is "Hello" (String)
```

```
BooleanSupplier booleanSupplier = () -> true;
booleanSupplier.getAsBoolean(); // The result is true (boolean)

IntSupplier intSupplier = () -> 33;
intSupplier.getAsInt(); // 33 (int)
```

9.5. OPTIONAL

Like many programming languages, Java uses null to represent the absence of a value. This approach may lead to exceptions such as NullPointerException and often reduces code readability due to explicit null checks. British computer scientist Tony Hoare – the inventor of the null concept – even described the introduction of null as a "billion-dollar mistake" because it led to countless bugs, security vulnerabilities, and system failures. To avoid problems associated with null, Java provides the Optional class, which is a safer alternative to the null standard.

The Optional class represents the presence or absence of a value of the specified type T. An object of this class can be empty or non-empty. Let's look at an example; in the following code snippet, two Optional objects are called absent and present. The first object represents an empty value (almost like null), and the second holds an actual string value.

```
Optional<String> absent = Optional.empty();
Optional<String> present = Optional.of("Hello");
```

`isPresent()` method checks whether an object contains anything:

```
System.out.println(absent.isPresent()); // false
System.out.println(present.isPresent()); // true
```

Starting with Java 11, we can also call it using the **`isEmpty()` method**. In situations where it's unknown whether a variable is null, it's better to use the `ofNullable` method instead of `of`. It produces an empty Optional if the value passed is null. In the following example, the `getRandomMessage` method might return null or some string message. Depending on what is returned, the result will be different.

```
String message = getRandomMessage();
Optional<String> optMessage = Optional.ofNullable(message);
System.out.println(optMessage.isPresent()); // Returns true or
false
```

If the message is not null (e.g., “Hello”), the code will print true. Otherwise, it will print false because the Optional object is empty. In a sense, Optional is like a box containing either a value or nothing. It wraps a value or represents the absence of a value, allowing you to check it using special methods.

The most obvious thing to do with an Optional is to get its value. Now, we will discuss three methods for that purpose:

- **get()** returns the value if it is present, otherwise it throws an exception;
- **orElse()** returns the value if it is present, otherwise it returns other;
- **orElseGet()** returns the value if it is present; otherwise, it calls other and returns its result.

Let's see how they work. First, use the get method to get the current value:

```
Optional<String> optName = Optional.of("John");
String name = optName.get(); // "John"
```

This code executes correctly and returns the name “John” from the Optional object. However, if an Optional object is empty, the program will throw a NoSuchElementException.

```
Optional<String> optName = Optional.ofNullable(null);
String name = optName.get(); // Throws a NoSuchElementException
```

This behavior is undesirable for a class intended to reduce runtime exceptions. Since Java 10, the preferred alternative to the get method is the orElseThrow method, which has similar behavior, but the name describes it better.

The orElse method is used to extract the value wrapped inside an Optional object or return a default value when Optional is empty. The default value is passed to the method as its argument:

```
String nullableName = null;
String name =
Optional.ofNullable(nullableName).orElse("unknown");
System.out.println(name); // unknown
```

The orElseGet method is quite similar, but it requires a supplier function to generate a result instead of taking some value to return:

```
String name = Optional
      .ofNullable(nullableName)
      .orElseGet(SomeClass::getDefaultResult);
```

Additionally, there are convenient methods that take functions as arguments and perform some action on the values wrapped inside Optional:

- **ifPresent** performs the given action with the given value; otherwise, it does nothing.
- **ifPresentOrElse** performs the given action with the given value; otherwise, it performs the action based on the given empty case.

The `ifPresent` method allows some code to be executed on the value if the `Optional` value is not empty. The method takes a consumer function that can handle the value.

The following example prints the length of the company name using an ifPresent statement.

```
Optional<String> companyName = Optional.of("Google");
companyName.ifPresent((name)                              ->
System.out.println(name.length())); // 6
```

However, the following code doesn't print anything because the Optional object is empty.

```
Optional<String> noName = Optional.empty();
noName.ifPresent((name)                                   ->
System.out.println(name.length()));
```

The "classic" equivalent of these two code snippets looks like this:

```
String companyName = ...;
if (companyName != null) {
```

```
        System.out.println(companyName.length());
}
```

The `ifPresentOrElse` method is a safer alternative to a full if-else statement. It executes one of two functions depending on whether the value is in `Optional`.

```
Optional<String> optName = Optional.ofNullable(/* some value
goes here */);

optName.ifPresentOrElse(
        (name) -> System.out.println(name.length()),
        () -> System.out.println(0)
);
```

Consider the example below; this is a case where Optional is not used, causing the program to throw an exception.

```
public static void main ( String [] args) throws IOException {
String[] str = new String[ 10 ];
   String lowercaseString = str [ 5 ].toLowerCase();
   System.out .print( lowercaseString );

}
```

Results after running:

```
Exception in thread "main" java.lang.NullPointerException:
Cannot invoke "String.toLowerCase()" because "str[5]" is null
        at Program.main(Program.java:11)
```

The above problem can be solved by using conditional expressions in combination with optionals for checking:

```
public static void main ( String [] args) throws IOException {
String[] str = new String[ 10 ];
   Optional <String> checkNull = Optional.ofNullable ( str [ 5
]);
   if ( checkNull .isPresent()){ // check for value is present
or not
        String lowercaseString = str [ 5 ].toLowerCase();
```

```
        System.out .print( lowercaseString );
else
System.out.println ( "Value does not exist " ) ;
}
```

Result:

```
```

```
public static void main ( String [] args) throws IOException {
String[] str = new String[ 10 ];
   str [ 5 ] = "Example of Optional" ;
   Optional <String> checkNull = Optional.ofNullable ( str [ 5
]);
if(checkNull.isPresent()){
        String lowercaseString = str [ 5 ].toLowerCase();
        System.out .print( lowercaseString );
else
System.out.println ( "Value does not exist " ) ;
}
```

Results after running:

```
Examples of optionals
```

Here's another example:

```
String[] str = new String[ 10 ];
str[ 5 ] = "Example of Optional";
Optional<String> empty = Optional.empty();
System.out.println ( empty);
Optional<String> value = Optional. of (str[ 5 ]);
System.out.println( "Result: " +value.filter((s)->s.equals(
"Abc" )));
System.out.println ( "Result: " + value.filter((s)->s.equals( "
EXAMPLE OF OPTIONAL" )));
System.out.println ( "Result: " + value.get ( ));
System.out.println ( "Value: " + value.hashCode ());
System.out.println ( " Result: " + value.isPresent());
```

```
System.out.println( "Result: " +Optional. ofNullable (str[ 5
]));
System.out.println( "Result: " +value.orElse( "Value is not
present" ));
System.out.println( "Result: " +empty.orElse( "Value is not
present" ));
value.ifPresent(System.out::println);
```

The following example will illustrate the use of Optional through several code snippets to demonstrate text data processing. It begins by creating a stream to read data from a text file:

```
public class OptionalTest {
   public static void main ( String [] args) throws IOException
{
       var contents = Files. readString (Paths. get (
"alice30.txt" ));
       List <String> wordList = List.of ( contents .split( "
\\ PL+" ));
}
}
```

Search for long words (words longer than 12 characters):

```
Optional <String> optionalValue = wordList.stream().filter(s ->
s.length()> 12 ).findFirst();
System.out.println(optionalValue.orElse( "No word" ));
```

Use the orElseGet() method in branching:

```
Optional<String> optionalValue = wordList.stream().filter(s ->
s.length()> 22 ).findFirst();
System.out.println(optionalValue.orElseGet(() -> {
String s = "Very long word" ;
return s;
}));
```

Use `ifPresent()` to check for existence instead of using the regular `if` syntax:

```
Optional<String> optionalValue = wordList.stream().filter(s ->
s.length()> 12 ).findFirst();
LinkedList<String> results = new LinkedList<String>();
```

```
optionalValue.ifPresent(v-> results .add(v));
System.out.println(results.getFirst());
```

Use the reference method:

```
var results = new HashSet<String>();
optionalValue.ifPresent(results::add);
```

Switch to map:

```
Optional<String> optionalValue = wordList.stream().filter(s ->
s.length()> 12 ).findFirst();
Optional<String> transformed =
optionalValue.map(String::toUpperCase);
System.out.println(optionalValue.orElse( "No word" ));
```

9.6. Stream API

9.6.1. Java 8 Stream

Java provides a new add-on package in Java 8 called java.util.stream. This package includes classes, interfaces, and enums to allow manipulation of data elements. Streams have several key characteristics:

- A stream does not store elements. It simply transmits elements from a source, such as a data structure, array, or I/O channel, through a path of computational operations.
- Streams follow a functional programming paradigm; operations on a stream do not modify the underlying data source. For example, filtering a stream obtained from a collection will create a new stream without the filtered elements, rather than removing the elements from the source collection.
- Streams are lazy and evaluate operations only when a terminal operation is invoked.
- Elements of a stream can only be accessed once during the stream's lifetime. Similar to an Iterator, a new stream must be created to access the same source elements again.

Streams can be used for filtering, collecting, printing, and converting

from one data structure to another, etc. By using the Stream API, a programmer doesn't need to write explicit loops, as each stream has an optimized loop internally.

The content above mentions sequentially processing objects using loops and collections. However, this method is quite prone to errors due to mutable variables and complex loop logic. It can also make the code harder to read, leading to increased complexity during further development. A key motivation for the new approach is to enable Java compilers to "parallelize" a computation, meaning dividing it into multiple parts that can run concurrently on several processors.

Conceptually, a stream resembles a collection, but it does not store elements. Instead, it retrieves elements from a source such as a collection, generator function, file, I/O channel, another stream, or something else, and processes them using a series of predefined operations, all combined into a single pipeline. Importantly, a stream pipeline has at most one terminal operation and an arbitrary number of intermediate operations.

There are three stages to working with a stream:

- Get the stream from a source.
- Perform intermediate operations with the stream to process the data.
- Perform the final step to produce the result.

9.6.2. Examples of Loops vs. Streams

All classes associated with streams are located in the java.util.stream package. There are several common stream classes: Stream<T>, IntStream, LongStream, and DoubleStream. While common streams work with reference types, others work with their respective primitive types. Consider the following simple example. Suppose we have a list of numbers and we want to count the numbers greater than 5:

```
List<Integer> numbers = List.of(1, 4, 7, 6, 2, 9, 7, 8);
```

A "traditional" way to do that is to write a loop like this:

```
long count = 0;
for (int number : numbers) {
      if (number > 5) {
      count++;
      }
}
System.out.println(count); // 5
```

This code prints "5" because the original list only contains five numbers greater than 5 (7, 6, 9, 7, 8). A loop with a filtering condition is a commonly used construct in programming. This code can be simplified by rewriting it using a stream:

```
long count = numbers.stream()
      .filter(number -> number > 5)
      .count(); // 5
```

Here, we get a stream from the list of numbers, then filter its elements using Predicate expressions, and then count the numbers that satisfy the condition. Although this code produces the same result, it is easier to read and modify. For example, we can easily change it to omit the first four numbers from the list.

```
long count = numbers.stream()
      .skip(4) // skip 1, 4, 7, 6
      .filter(number -> number > 5)
      .count(); // 3
```

Stream processing is performed as a sequence of method calls separated by dots with a single terminal operation. For easy reading of data processing within a stream, each method call should be placed on a new line if the line contains more than one operation.

9.6.3. Creating Streams

There are many ways to create streams, including using lists, sets, strings, arrays, etc., as sources. The most common way to create a stream is to get it from a Collection. Any collection has a stream() function for this purpose.

```
List<Integer> famousNumbers = List.of(0, 1, 1, 2, 3, 5, 8, 13,
21, 34, 55);
Stream<Integer> numbersStream = famousNumbers.stream();
Set<String> usefulConcepts = Set.of("functions", "lazy",
"immutability");
Stream<String> conceptsStream = usefulConcepts.stream();
```

It is also possible to get a stream from an array:

```
Stream<Double> doubleStream = Arrays.stream(new Double[]{ 1.01,
1d, 0.99, 1.02, 1d, 0.99 });
```

Create a live stream from some values:

```
Stream<String> persons = Stream.of("John", "Demetra",
"Cleopatra");
```

Create a stream by joining different streams together:

```
Stream<String> stream1 = Stream.of(/* some values */);
Stream<String> stream2 = Stream.of(/* some values */);
Stream<String> resultStream = Stream.concat(stream1, stream2);
```

There are several ways to create empty streams (which can be used as return values from methods):

```
Stream<Integer> empty1 = Stream.of();
Stream<Integer> empty2 = Stream.empty();
```

Additionally, there are other methods for creating streams from various sources: from files, from I/O streams, etc.

9.6.4. Operations on Streams

All stream operations are divided into two groups: intermediate operations and terminal operations.

- **Intermediate operations** are not evaluated immediately upon calling. They simply return new streams to call subsequent operations on them. Such operations are called lazy because they don't actually do anything useful.
- **Terminal operations** initiate all evaluations with the stream to produce results or generate side effects. As mentioned previously,

a stream pipeline must have at most one terminal operation.

Once the terminal operation has evaluated, the stream cannot be reused. Attempting to do so will throw an IllegalStateException. Streams provide a large number of operations that can be performed on elements. Some typical operations will be presented in the next section of this course.

9.6.5. Intermediate Operations

Methods involving intermediate operations:

- `filter()` returns a new stream containing elements that match a predicate;
- `limit()` returns a new stream containing the first `n` elements of the stream;
- `skip()` returns a new stream without the first `n elements of the stream;`
- `distinct()` returns a new stream containing only the unique elements resulting from `equals` ;
- `sorted()` returns a new stream containing elements sorted in the natural order or **comparator** passed to it;
- `peek()` returns the same stream of elements but allows you to observe the current elements of the stream for debugging;
- `map()` returns a new stream containing the elements obtained by applying a function (i.e., transforming each element).

9.6.6. Terminal Operations

Terminal Operations:

- `count()` returns the number of elements in the stream as a `long value`;
- `max()`/`min()` returns the optional largest/smallest element of the stream according to the given comparator;

- `reduce()` combines values from a stream into a single value (a composite value);
- `findFirst()` / `findAny()` returns the first/any element of the stream as `Optional` ;
- `anyMatch()` returns `true` if at least one element matches a Predicate (see also: `allMatch` , `noneMatch`);
- `forEach` takes a **consumer** and applies it to each element of the stream (e.g., prints it);
- `collect()` returns a collection of values in the stream;
- `toArray()` returns an array of values in a stream.

Operations (functions) such as `filter`, `map`, `reduce`, `forEach`, `anyMatch`, and several others are called higher-order functions because they accept other functions as arguments. Some terminal operations return `Optional` because the stream can be empty and need to specify a default value or an action if the stream is empty.

9.7. PARALLEL STREAMS

One of the biggest advantages of the Stream API and functional programming in general is the ability to easily write clear code to handle data in parallel. There's no need to manually create threads, check if the code is properly synchronized, and call **wait/notify methods**. All of this is done automatically within the parallel streams.

There are several simple ways to create parallel streams:

- `parallelStream()` method of a collection instead of `stream()` :

```
List<String> languages = List.of("java", "scala", "kotlin",
"C#");

List<String> jvmLanguages = languages.parallelStream()
      .filter(lang -> !Objects.equals(lang, "C#"))
      .collect(Collectors.toList());
```

```
System.out.print(jvmLanguages); // [java, scala, kotlin]
```

- Convert an existing stream into a parallel stream using the `parallel() method` :

```
long sum = LongStream
      .rangeClosed(1, 1_000_000)
      .parallel()
      .sum();

System.out.println(sum); // 500000500000
```

It's actually easy to create a parallel stream, but should we always do this? Not necessarily; a parallel stream isn't always faster than its sequential equivalent. Several factors significantly affect the performance of a parallel stream. The number of CPU cores available at runtime significantly affects performance; more cores generally lead to better performance.

The cost per processing element is another important factor. The longer each element takes to process, the more efficient parallelization becomes, but parallel processing should not be used for excessively long operations (e.g., network connectivity).

9.7.1. The reduce method and its initial value

For example, suppose you want to calculate the sum of numbers and add 100 to the result. When using sequential stream, simply set 100 as the initial value (seed) of the `reduce() method` :

```
int result = numbers.stream().reduce(100, Integer::sum);
```

This code produces the same result as

```
int result = numbers.stream().reduce(0, Integer::sum) + 100;
```

However, when using a parallel stream, the first piece of code will produce an incorrect result. The reason is that the dataset will be divided into several parts, and the value 100 will be added to each part. When using a parallel stream, only use a neutral element (0 for sum, 1 for multiplication, etc.) as the initial value in the `reduce method` .

9.7.2. Traversing and Sorting Elements

Given a sorted list of numbers 1, 2, ..., 10. Process and print each number from the list using streams.

This is a sequential stream:

```
sortedNumbers.stream()
      .map(Function.identity()) // some processing
      .forEach(n -> System.out.print(n + " "));
```

The output is:

```
1 2 3 4 5 6 7 8 9 10
```

This is a parallel stream:

```
sortedNumbers.parallelStream()
      .map(x -> x) // some processing
      .forEach(n -> System.out.print(n + " "));
```

Output:

```
6 7 9 8 10 3 4 1 5 2
```

9.7.3. Notes on the order of parallel streams

`forEach` method breaks the order when used with parallel streams. If this is rewritten using `forEachOrdered` , the code will work as expected:

```
sortedNumbers.parallelStream()
      .map(Function.identity()) // some processing
      .forEachOrdered(n -> System.out.print(n + " "));
```

When using a parallel stream, use `forEachOrdered` instead of `forEach` if the order of the elements is important. However, this will also slow down the parallelization process.

Consider another example, suppose you want to extract the first three even numbers from a parallel stream from two coupled streams.

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
```

```
List<Integer> emptyList = List.of();
```

This is how to merge and process two lists.

```
Stream.concat(numbers.stream(), emptyList.stream())
      .parallel()
      .filter(x -> x % 2 == 0)
      .limit(3)
      .forEachOrdered((n) -> System.out.print(n + " "));
```

Output:

```
2 4 6
```

But if you create an empty list using `Collections.emptyList()` , the result will be:

```
2 4 10
```

Therefore, caution is needed when using parallel streams. Obviously, Stream makes using parallel streams very easy. But they are not always faster than their sequential equivalents because their performance depends on many factors including data volume, hardware, and the operations used. At the same time, it is quite difficult to predict speed increases without performing measurements in practice. Additionally, there are some potential drawbacks when using a particular parallel stream related to the order of elements. Therefore, it is necessary to understand why parallel streams are needed in different situations. If there are sufficient resources or the operations are simple and do not require high speed, it may be better to use sequential streams. But if stable speeds have been achieved and a clear effect is observed, parallel streams can be tried to improve speed.

QUIZZES

Which Java types can Lambda expressions be used with?	
Interface	
Class	

Abstract class	
All of them are correct.	

How many parameters can a lambda expression have?	
One	
Two	
Three	
Unlimited	

In which version of Java were lambda expressions introduced?	
Java 7	
Java 8	
Java 9	
Java 10	

What is the purpose of using Lambda expressions in Java?	
Write more concise code	
Increase program execution speed	
Reduce the size of the code file	
Both A and B	

What are the components of a Lambda expression in Java?	
Parameters and body	
Parameters, lambda expressions, and return values	
Lambda expressions and return values	
Return value only	

What are functional interfaces used for in Java?	
To mark an interface as abstract	
To define abstract methods	
To define methods that can use Lambda.	

To define methods that can be used with anonymous classes.	

What is the syntax for using Lambda with Functional Interfaces?	
(parameters) -> expression	
(parameters) -> { statement }	
(parameters) -> { return expression; }	
All of them are correct.	

In Method Reference, what syntax is used to refer to a method of a specific object?	
site::method	
ClassName::method	
ClassName::new	
country::new	

What are optionals in Java?	
A data type	
An interface	
A class	
A function	

What are the main benefits of Optional?	
Avoid NullPointerException	
Increase processing speed	
Create more readable source code.	
Ensuring data integrity	

Which method is used to filter elements of a stream based on a condition provided by a Predicate object?	
filter()	
map()	
reduce()	
forEach()	

Which method is used to perform an operation on each element of a Stream and return a new Stream containing the result?	
filter()	
map()	
reduce()	
forEach()	

What method is used to reduce a collection of Stream elements to a single element by applying a specific operation to the elements?	
filter()	
map()	
reduce()	
forEach()	

PRACTICE

Lesson 1. Creating and starting threads

This exercise demonstrates how Lambda expressions can be used to simplify code:

```
public static void main ( String [] args) {
   Runnable r1 = new Runnable () {
public void run () {
System.out.println( "Thread1 is running..." );
}
};
   Thread t1 = new Thread( r1 );
t1.start();

   //Thread Example with lambda
   Runnable r2 = () -> {
       System.out.println( "Thread2 is running..." );
};
   Thread t2 = new Thread( r2 );
```

```
t2.start();
}
```

Lesson 2. Sorting Products

The following exercise illustrates the use of **Lambda** to shorten code when comparing and sorting:

```
import java.util.ArrayList ;
import java.util.Collections ;
import java.util.List ;

class Product {
   int id ;
   String name ;
   int price ;

   public Product ( int id, String name, int price) {
        super ();
        this.id = id ;
        this.name = name ;
        this.price = price ;
}
}

public class Program {
   public static void main ( String [] args) {
List<Product> list = new ArrayList<Product>();

        //Adding Products
        list.add( new Product( 1 , "Iphone 14" , 850 ));
list.add( new Product( 3 , "Samsung Note21" , 300 ));
list.add( new Product( 2 , "Vivo 7 Plus" , 150 ));

        System.out.println( "Sorting on the basis of price..."
);
```

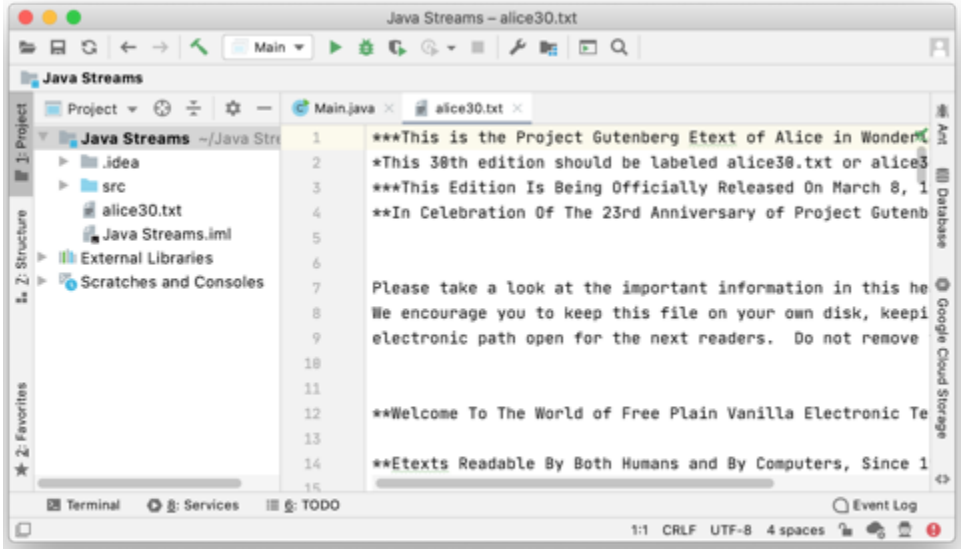
```
        Collections. sort (list, (o1, o2) -> ( int ) (o1. price
- o2. price ));


        for (Product p : list) {
System.out.println(p.id + " " + p.name + " " + p.price ) ;
}
}
}
```

Lesson 3. Text Data Processing

Download the data file from the address below and place it in the project folder:

https://raw.githubusercontent.com/mthli/Java/master/CoreJava/gutenberg/alice30.txt

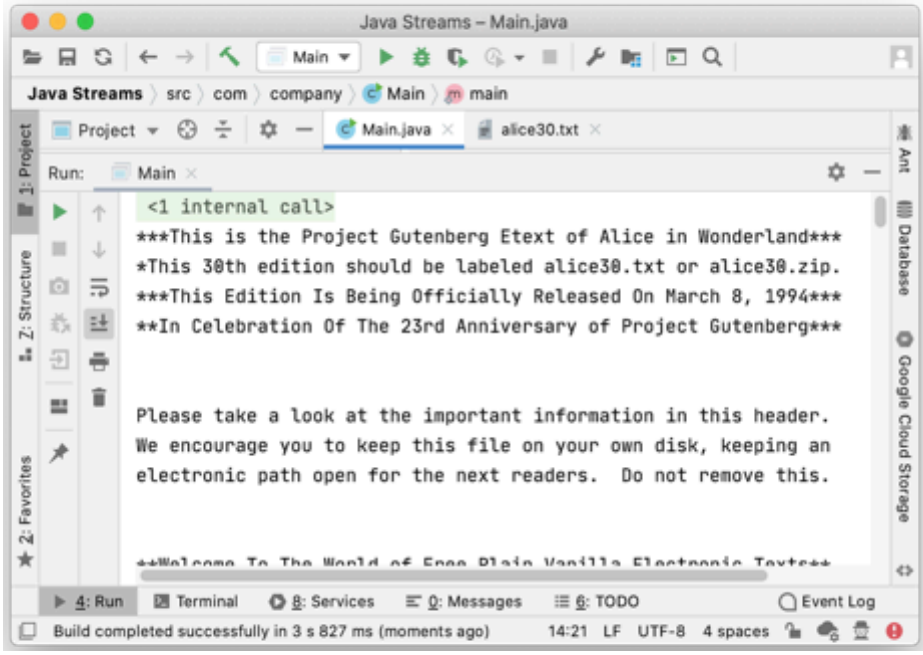


Write code to read the above-mentioned file content:

```
public static void main ( String [] args) {
try {
File myObj = new File( "alice30.txt" );
        Scanner myReader = new Scanner( myObj );
while (myReader.hasNextLine()) {
```

```
String data = myReader.nextLine();
System.out.println ( data);
}
myReader.close();
} catch ( FileNotFoundException e) {
System.out.println ( "An error occurred." ) ;
e.printStackTrace();
}
}
```

The results are displayed:



Use streams to count long words (words with more than 12 characters):

```
public static void main ( String [] args) {
   String contents ;
try {
contents = Files. readString (Paths. get ( "alice30.txt" ));
       List <String> words = List.of ( contents .split( " "
));
```

```
long count = 0 ;
       for ( String w : words ) {
if (w.length() > 12 ) count++;
}
System.out.println ( count);
} catch ( IOException e) {
e.printStackTrace();
}
}
```

The result shows the number of characters in the text as 947:

```
947
```

If using Stream, the syntax is:

```
public static void main ( String [] args) {
   String contents ;
try {
contents = Files. readString (Paths. get ( "alice30.txt" ));
       List <String> words = List.of ( contents .split( " "
));
long count = words.stream().filter(w -> w.length() > 12
).count();
System.out.println ( count);
} catch ( IOException e) {
e.printStackTrace();
}
}
```

The result is similar to traditional counting methods:

```
947
```

Counting long characters using parallel stream technology.

```
public static void main ( String [] args) {
   String contents ;
try {
contents = Files. readString (Paths. get ( "alice30.txt" ));
```

```
        List <String> words = List.of ( contents .split( " "
));
long count = words.parallelStream().filter(w -> w.length() > 12
).count();
System.out.println ( count);
} catch ( IOException e) {
e.printStackTrace();
}
}
```

Output:

```
947
```

Lesson 4. Filtering data using Streams

Implement a list and filter the necessary data from that list using standard methods (traversal, iteration):

```
import java.util. *;
class Product {
   int id ;
   String name ;
   float price ;
   public Product ( int id, String name, float price) {
        this.id = id ;
        this.name = name ;
        this.price = price ;
}
}
public class Program {
   public static void main ( String [] args) {
        List < Product > productsList = new ArrayList< Product
>();
        //Adding Products
        productsList.add( new Product( 1 , "HP Laptop" , 25000f
));
productsList.add( new Product( 2 , "Dell Laptop" , 30000f ));
productsList.add( new Product( 3 , "Lenovo Laptop" , 28000f ));
```

```
productsList.add( new Product( 4 , "Sony Laptop" , 28000f ));
productsList.add( new Product( 5 , "Apple Laptop" , 90000f ));
        List < Float > productPriceList = new ArrayList< Float
>();
        for ( Product product : productsList ){

            // filtering data of list
            if ( product.price < 30000 ){
                productPriceList .add( product.price ); //
adding price to a productPriceList
            }
}
System.out.println(productPriceList); // displaying data
    }
}
```

Re-implement the above task using Stream:

```
import java.util.ArrayList ;
import java.util.List ;
import java.util.function.Function ;
import java.util.function.Predicate ;
import java.util.stream.Collectors ;

class Product {
    int id ;
    String name ;
    float price ;

    public Product ( int id, String name, float price) {
        this.id = id ;
        this.name = name ;
        this.price = price ;
}
}

public class Program {
```

```
   public static void main ( String [] args) {
        List < Product > productsList = new ArrayList< Product
>();
        productsList.add( new Product( 1 , "HP" , 25000f ));
productsList.add( new Product( 2 , "Dell" , 30000f ));
productsList.add( new Product( 3 , "Lenovo" , 28000f ));
productsList.add( new Product( 4 , "Sony" , 28000f ));
productsList.add( new Product( 5 , "Apple" , 90000f ));
        List < Float > productPriceList2 = productsList
.stream()
.filter( new Predicate<Product>() {
                    @Override
                    public boolean test ( Product p) {
                        return p. price > 30000 ;
}
}) // filtering data
                .map( new Function < Product , Float >() {
                    @Override
                    public Float apply ( Product p) {
                        return p. price ;
}
}) // fetching price
                .collect( Collectors .toList()); // collecting
as list
        System.out.println( productPriceList2 );
}
}
```

Use Lambda to shorten statements:

```
import java.util.ArrayList ;
import java.util.List ;
import java.util.function.Function ;
import java.util.function.Predicate ;
import java.util.stream.Collectors ;

class Product {
```

```
    int id ;
    String name ;
    float price ;

    public Product ( int id, String name, float price) {
        this.id = id ;
        this.name = name ;
        this.price = price ;
}
}

public class Program {
    public static void main ( String [] args) {
        List < Product > productsList = new ArrayList< Product
>();
        //Adding Products
        productsList.add( new Product( 1 , "HP" , 25000f ));
productsList.add( new Product( 2 , "Dell" , 30000f ));
productsList.add( new Product( 3 , "Lenovo" , 28000f ));
productsList.add( new Product( 4 , "Sony" , 28000f ));
productsList.add( new Product( 5 , "Apple" , 90000f ));
        List < Float > productPriceList2 = productsList
.stream()
.filter(p -> p. price > 30000 ) // filtering data
                .map(p -> p.price ) // fetching price
                .collect( Collectors .toList()); // collecting
as list
        System.out.println( productPriceList2 );
}
}
```

Lesson 5. Practice creating streams

Create a stream from the data in the file:

```
public static void main ( String [] args) throws IOException {
    Path path = Paths.get ( "alice30.txt" );
    var contents = Files. readString (path);
```

```
   Stream <String> words = Stream.of ( contents .split( " \\
PL+" ));
}
```

Create a `show` method to print the data the stream contains:

```
public static void show (String title, Stream<String> stream) {
   System. out .print(title + ": " );
stream.limit( 10 ).collect(Collectors. toList ()).forEach(s ->
System. out .print(s + "," ));
   System.out.println ( );
}
```

Call the show method from the main() function:

```
show ( "words" , words );
```

The result displayed when the function is called:

```
words:
***This,is,the,Project,Gutenberg,Etext,of,Alice,in,Wonderland
***
This,
```

Create a stream from "words":

```
Stream <String> name = Stream.of ( "Dai" , "hoc" , "kinh" ,
"te" , "quoc" , "dan" );
show ("name", name);
```

Create an empty stream:

```
Stream <String> silence = Stream.empty ();
show ( "silence" , silence);
```

Result:

Create an "echo" stream. This is a stream that produces the same content, for example:

```
Stream <String> echos = Stream.generate (() -> "KTQD" );
show ( "echos" , echoes);
```

Result:

```
echoes: KTQD, ...
```

Create a stream containing random data; in this case, use a function reference to generate random numbers:

```
Stream < Double > randoms = Stream.generate ( Math:: random );
show ( "Random" , randoms);
```

The show function can be rewritten using generic programming:

```
public static < T > void show (String title, Stream< T >
stream) {
   System. out .print(title + ": " );
stream.limit( 10 ).collect(Collectors. toList ()).forEach(s ->
System. out .print(s + "," ));
   System.out.println ( );
}
```

The results are displayed:

```
Random:
0.807718927722843,0.36946784644216246,0.08143227201572512,0.6
446576003097284,0.366443263082484,0.9752767239380946,0.618858
7519876564,0.7061125729225125,0.8214336091276199,0.6824309847
153482,
```

PRACTICE EXERCISES

Lesson 1. Practice with Lambda

- Write a Java program that uses Lambda expressions to sort a list of strings alphabetically.
- Write a Java program that uses Lambda expressions to calculate the sum of numbers in a list of integers.
- Write a Java program that uses Lambda expressions to find the largest number in a list of integers.

Lesson 2. Practice with the Stream API

- Write a program that uses the Stream API to find even numbers in a list of integers.

- Write a program that uses the Stream API to find strings with a length of 5 or more in a list of strings.
- Write a program that uses the Stream API to find prime numbers in a list of integers.

REFERENCES

[1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).

[2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.

[3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.

[4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.

[5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.

[6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.

[7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.